Dynamic programming exercises

A05-C01 | What history must a state retain?

A rod has integer length 4, prices $p[1..4] = [1, 5, 8, 9]$, and free cuts. All material must be sold, with at most 2 pieces. History A has sold one length-2 piece; history B has sold two length-1 pieces. Both leave length 2. Is remaining length a sufficient state? Define a sufficient state, bases, and recurrence.

A05-C02 | Fill the table and resolve ties

Let $n = 4$ and $p[1..4] = [2, 5, 7, 8]$. Use all material; cuts are free. Process both j and first-piece candidates i in increasing order; retain smaller i on ties. List every row's candidates, $r[j]$, and first[j], then reconstruct an optimum.

A05-C03 | Why a proof needs both directions

Every positive integer piece length has a price. Cuts are free and all material must be sold. Set $r[0] = 0$ and $r[j] = \max_{1 \le i \le j}(p[i] + r[j - i])$. Assuming shorter states are correct, prove both $OPT(j) \le r[j]$ and $r[j] \le OPT(j)$. Which feasible solutions are omitted if only $i < j$ is allowed?

A05-C04 | Dependencies and repeated calls

Use the preceding recurrence with target $n = 4$. Naive recursion calls every $j - i$ in order; each base call at $j = 0$ also counts. Find the distinct state count, dependency DAG edge count, total naive calls, and the number of nonbase states expanded and candidates evaluated with memoization. Generalize each count to n.

A05-C05 | An invalid longest-path substitution

An undirected graph has $V = \{a, b, c, d\}$, $E = \{ab, bc, cd, da\}$, and unit edge lengths. Paths may not repeat vertices. In $a \to b \to c \to d$, find a suffix that is not longest between its endpoints and its independently optimal replacement. Write the substituted sequence and explain its infeasibility. Define a set state, recurrence, and evaluation measure that repair the problem. Is the resulting time necessarily polynomial?

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A05-C06 | Negative prices and cut charges

All pieces must be sold; discarding is forbidden. (a) For $n = 2, p = [-2, -5]$, compute every state and reconstruct a solution. Explain why initializing each maximum to 0 fails. (b) For $n = 4, p = [1, 5, 8, 9]$, charge 2 per actual cut. Give the recurrence, $r[0..4]$, and an optimal decomposition. (c) What is returned for $n = 0$ with an empty price table?

A05-K01 — Complete table and subset

For 0/1 knapsack with capacity 5, items in order are $(w, v) = (2, 10), (1, 6), (3, 12)$; the empty set is allowed. Let $D[i, c]$ optimize the first i items within capacity c. Fill every cell for $0 \leq i \leq 3, 0 \leq c \leq 5$, then recover one solution, skipping an item on ties.

A05-K02 — The missing item index

Capacity is 2 and there is one item $(w, v) = (1, 4)$, available once. A proposed recurrence is $F(c) = \max(0, 4 + F(c - 1))$, with $F(0) = 0$. Compute $F(2)$, identify the error, and give a correct state explaining why taking the item removes its availability.

A05-K03 — Choose direction from the invariant

One item has $(w, v) = (2, 3)$, $W = 4$, and initially $dp = [0, 0, 0, 0, 0]$. Apply $dp[c] \leftarrow \max(dp[c], 3 + dp[c - 2])$ in ascending order $2, 3, 4$ and descending order $4, 3, 2$. Show the full array after each step. Which solves 0/1, and which solves unbounded knapsack?

A05-K04 — Exact weight and negative values

The 0/1 items are $(2, -3), (3, 4)$ and capacity is 5. Find optima and subsets for weight at most 5, exactly 5, and exactly 1. Give the initial and final rows of an exact-weight DP for capacities 0 through 5, and explain why an all-zero initialization is wrong.

A05-K05 — Ties and reconstruction

The 0/1 items are $(1, 2), (1, 2)$ and $W = 1$. Give the complete table and trace back by skipping on ties. Is the optimum unique? Can the same two-dimensional comparisons be made from only the final one-dimensional values? Identify the missing information.

A05-K06 — Greedy failure and a valid bound

Items are $(2, 10)$, $(1, 6)$, $(3, 12)$, capacity is 4, and all values are positive. Compute the 0/1 density-greedy result, the true 0/1 optimum, and the fractional optimum, listing each selection. Why is the fractional value a valid branch-and-bound upper bound? With incumbent 18, can nodes bounded by 17 and by 20 be pruned, when one optimum suffices?

A05-K07 — Input length and boundaries

Capacity is $W = 2^b$ and there are n positive-integer-weight items. Why does $O(nW)$ not give a general polynomial bound in binary input length? Also give the 0/1 at-most answer for $W = 0$, for all-negative values, and the unbounded-knapsack outcome if $(w, v) = (0, 1)$ is allowed.

Sequence exercises

A05-S01

For $X = \texttt{ABAC}$ and $Y = \texttt{BACA}$, compute the complete LCS-length table including zero boundaries. Give one LCS and its index embeddings in both inputs.

A05-S02

For $X = \texttt{A}$ and $Y = \texttt{AA}$, evaluate: “When final symbols agree, every optimal matching must pair the two final positions.” Distinguish a string from an index embedding and prove the correct length recurrence.

A05-S03

For $X = \texttt{ABCBDAB}$ and $Y = \texttt{BDCABA}$, trace from $(7, 6)$ to a boundary: equal symbols move diagonally; unequal symbols compare above and left, preferring above on ties. List all coordinates and recorded symbols. Give two other optimal strings with valid embeddings.

A05-S04

Compute the LCS length of $X = \texttt{ABA}$ and $Y = \texttt{BAA}$ using two rows. List each completed row. Explain the missing information if someone attempts ordinary full-table traceback from the last two rows, and give two remedies.

A05-S05

Globally align $X = \texttt{ACGTA}$ and $Y = \texttt{AGCAC}$ with match +3, mismatch −2, and symbol/gap −3; double-gap columns are forbidden. Give the complete F table, one optimal alignment, and its column scores. Why is presenting a feasible score-1 alignment alone not an optimality proof?

A05-S06

Globally align $X = \texttt{A}$ and $Y = \texttt{C}$ with match +3, mismatch −2, gap −3. Compute $F[1, 1]$ with correct boundaries and with incorrect all-zero boundaries. Repeat for $X = \texttt{A}$ and $Y = \texttt{CA}$. Explain why the first test misses the bug.

A05-S07

For $X = \texttt{ABAC}$ and $Y = \texttt{BACA}$, compare LCS and longest common substring. Give optimal lengths and witnesses. Define a substring state $H[i, j]$ required to end at both positions, state its boundary/equal/unequal recurrences, and identify where the final answer is found.

A05-S08

Prove the three-branch recurrence for global alignment with a linear gap cost on inputs of lengths $m, n > 0$. Include an upper bound and achievability. State time/space costs for filling, full-table reconstruction, and score-only computation. Explain why the algorithm should not compare only the diagonal whenever symbols match.

A05-I01 Complete matrix-chain calculation

For $p = [5, 10, 3, 12, 5]$, fill every $m[i, j]$ by increasing length. List every split candidate for every non-singleton, record s, and reconstruct parentheses and all intermediate dimensions.

A05-I02 The cheapest merge alone is insufficient

For $p = [3, 2, 4, 100]$, someone chooses the smallest final merge cost $p_0 p_k p_3$. Compute both complete candidates and determine whether this rule is optimal.

A05-I03 Proof and exact candidate counts

For the basic matrix-chain DP with $n \geq 1$ matrices, prove that replacing a child solution preserves final multiplication dimensions. Derive $(n^3 - n)/6$ candidates. Evaluate $n = 4$ and explain why this differs from the five full parenthesizations.

A05-I04 Full OBST example B

For $k_1 < k_2 < k_3$, $p = (.2, .3, .1)$, $q = (.1, .05, .15, .1)$, use convention E. List empty costs, every interval mass and every candidate root. Reconstruct the optimum and directly audit C and E.

A05-I05 Diagnose mixed empty-cost conventions

With one key, $p_1 = .6, q_0 = .1, q_1 = .3$, A initializes empty costs to zero and reports 1.0; B initializes them to the corresponding q and reports 1.4. Who is wrong? Derive the general $E - C$ relation and explain why different values alone do not establish an error.

A05-I06 Global mass and conditional expectation

Use $p = (.2, .3, .1)$, $q = (.1, .05, .15, .1)$. Interval $[1, 1]$ has $E = .50$, $w = .35$, hence conditional expectation $.50/.35$. Why is adding this directly to root 2' s right cost .60 and root cost 1 invalid? Give the correct combination and the effect of multiplying all weights by 20.

A05-I07 Propagate ties through reconstruction

For four keys, $p = (3, 3, 1, 1)$, $q = (2, 3, 1, 1, 1)$. Given $C_{11} = 8, C_{22} = 7, C_{33} = C_{44} = 3, C_{12} = 19, C_{23} = 12, C_{24} = 19, C_{13} = 25$, empty costs 0, find all roots for C_{34} and all optimal trees for C_{14}. Convert global costs to expected C/E per query.

A05-I08 Exact OBST candidate and output bounds

Derive $n(n+1)(n+2)/6$ root candidates in basic OBST DP and evaluate $n = 3$. Explain why all tied roots can fit in $O(n^3)$ table space while outputting all optimal trees need not take polynomial time.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. Lecturer Charles E. Leiserson. MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. Lecturer Charles E. Leiserson; slides credited to Erik D. Demaine and Charles E. Leiserson. Original scanned algorithms notes (演算法_wjungle), dynamic programming pages; original scanned data structures notes (資料結構), physical pages 166–170, for comparison of optimal search tree models. These notes cover algorithms and data structures and are not the full text of a single textbook.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study